FoveaWAM: Role-Aware 4D World–Action Modeling for Dynamic Manipulation Supplementary Material

Anonymous CVPR submission

A. Method and Training Details

A.1. Architecture

Table S1 lists the configuration of every module, in the order of the data flow described below.

Table S1. Architecture of FoveaWAM. Heads are skipped at deployment.

Module	Configuration
Visual encoder	frozen MAE ViT-B/16, 224×224 input, 196 patch tokens
Target-state tokenizer	$28 \to 1024 \to 1024$ GELU MLP, additive relative-time embedding
Robot-state tokenizer	linear, $7 \to 1024$
Tokens	212 scene (196 patch, 8 target-state, 8 robot), 16 summary, 8 action
Backbone	24 layers, width 1024, 16 heads, shared attention, SiLU FFN 4096 per stream, 504.0M parameters
Pose head	linear, mean summary state $\to 8 \times 6$
Flow head	linear per patch token, $8 \times (3{+}1)$
Appearance heads	linear per patch token: depth and moving-surface RGB as 16×16 pixels, 1024-d DINOv3 ViT-L/16 features
Target reconstruction	$1024 \to 256 \to 3072$ GELU MLP with sigmoid, 32×32 RGB
Action stream	flow matching, adaptive layer norm on the flow time, 5 Euler steps

Scene tokens. Each patch token of the frozen MAE ViT-B/16 [6] adds a learned view embedding and linear embeddings of its mean valid metric depth and of the base-frame position $\mathbf{x}_i$ of its back-projected center pixel; a patch without valid depth at its center receives no position embedding. Only the patch queries and keys receive 3D rotary embeddings from $\mathbf{x}_i$, and the same points are the source points of the sparse 3D flow. Past images never enter the model.

State tokens. The 8 target-state tokens sample the offsets $-350, -300, \ldots, 0$ ms. Each is a 25-dimensional descriptor $(\mathbf{p}, \log R, \Delta\mathbf{p}, \log \Delta R, \Delta t, \mathbf{v}, \boldsymbol{\omega}, \mathbf{a}, \boldsymbol{\alpha})$: slots 0–6 describe the seven intervals between consecutive offsets, with velocities and accelerations from finite differences over the elapsed times, and slot 7 holds the current pose in the base frame. Each dimension is standardized with training statistics, invalid dimensions are zeroed, and three validity flags (pose, interval, acceleration) are appended; a slot without any valid feature is masked out of attention. The robot tokens hold the end-effector position, rotation vector, and gripper opening at the same offsets, and 16 learned embeddings form the summary tokens. The policy has no language input.

Backbone and heads. The mask of Sec. 3.4 sets blocked attention logits to $-\infty$ in every layer and head, in training and at deployment, so the world states are computed once per query and reused across the Euler steps. All heads read the final world states and are skipped at deployment; no token reads their outputs. The pose head reads the mean summary state ($\mathcal{L}_P$, Eq. 3). The flow head gives each patch point's displacement and visibility logit at every horizon ($\mathcal{L}_F$, Eq. 2, with scale $s = 0.7154605$ m and visibility weight 1); valid but invisible points still supervise their displacement. $\mathcal{L}_{\text{aux}}$ sums four losses: squared errors on future metric depth (valid pixels), on the RGB of surfaces that move by more than 1 mm, and on layer-normalized DINOv3 features [12], and an absolute error on the current target's RGB region reconstructed from the mean summary state.

Action stream. Each action token embeds a seven-dimensional command: a base-frame translation increment, a left-relative rotation vector, and a gripper command, relative to the measured end-effector pose at application. At deployment, the decoded commands are clipped to 0.10 m per translation axis and 0.5 rad per rotation axis, and the gripper changes state only when its command leaves a deadband of 0.5 in the ball-capture tasks and 0.1 in the others.

A.2. Training on ConMoto

Each control step of a demonstration anchors a window of 8 query positions, each with its own causal history and no attention across positions, giving 1,324,374 windows from the 11,200 demonstrations; 350 further episodes form the validation split. Flow labels follow the rendered surface

point at each patch center through the ground-truth rigid motion of its MuJoCo geometry; a future point is visible if it projects into the image onto the same geometry within a depth tolerance of $\max(1\,\mathrm{mm},\ 0.001z)$. Command j of query t is labeled with the demonstration command issued at $\tau_t + j\Delta_a$, since d_{task} applies only at execution (Eq. (S1)), and commands are standardized over the training split. Each query position averages the action loss over 8 noise–time pairs, and each loss is normalized by its number of valid labels in the global batch. Table S2 lists the recipe.

Table S2. Training hyperparameters on ConMoto.

Item	Value
Optimizer	AdamW, $\beta = (0.9, 0.999)$, $\epsilon = 10^{-8}$, weight decay 10^{-4}
Learning rate	2×10^{-4}, 5,174 warm-up updates from 0, cosine decay to 0
Batch, schedule	global batch 512 windows, at most 20 epochs, lowest-validation-loss checkpoint
Loss weights	$\lambda_F = \lambda_P = \lambda_R = 0.1$, $\lambda_{\mathrm{aux}} = 0.1$ per target
Action noise	8 noise–time pairs per query position
Precision	trainable weights FP32, frozen encoder BF16, gradient clipping at 0.1
Inference	FP32, world states cached within a query
Hardware	[N] RTX PRO 6000 GPUs, [X] hours per run

A.3. Pretraining on DROID and Fine-Tuning

The real-robot base trains on DROID [8] (15 Hz) with the first exterior camera and its depth, using the 3D point tracks and calibrations released by PointWorld [7], matched to the raw episodes by robot states, commands, and camera identity. The 42,087 usable episodes yield 598,889 anchor windows (538,740 for training, 60,149 for validation); each anchor expands into 8 query positions, and only the anchor carries flow and DINOv3 labels. All action slots with an issued command are kept, without a speed filter. Each patch follows one PointWorld track that starts inside it, preferring the one with the largest consistent motion so that small moving objects are covered; 47.05% of the point–horizon slots carry a label.

DROID provides no object poses, so the base has no target-state tokens or pose head; it reads the current RGB-D frame and the robot history and trains the action loss and the sparse 3D flow with weight 1 and the future DINOv3 features with weight 0.1, without the visibility loss. Since DROID objects mostly move only when the robot moves them, the base mainly learns metric geometry and self-induced motion; exogenous target motion enters during fine-tuning. The base has 48 layers at width 1024 (1.02B trainable parameters) and trains for 20 epochs (21,060 updates) at a global batch of 512 on 64 GPUs with the optimizer and precision of Table S2. Each real-robot policy is fine-tuned from it on its task's demonstrations [updates, learning rate, and batch size], with the target-state tokens and missing heads freshly initialized; no real-robot policy uses simulation data or weights.

B. ConMoto and Evaluation Protocol

B.1. Benchmark Design

ConMoto comprises fourteen tasks in five families (Fig. 3) that cover translation, rotation, temporary occlusion, constrained passage, and abrupt changes of the target's motion. We will release the tasks, scenarios, success checks, and demonstrations.

Environment. The tasks run in MuJoCo 3.11.0 and robosuite 1.5.2 [13, 18] with a Franka Panda arm and a parallel gripper under robosuite's operational-space pose controller, with a control period of 50 ms and a physics step of 2 ms. A fixed third-person camera at $(0.68, -0.58, 1.62)$ m with a 40° vertical field of view renders 224×224 RGB-D images. An action is an end-effector translation and rotation-vector increment in the base frame with a gripper command, clipped to 0.10 m per translation axis and 0.5 rad per rotation component.

Success and termination. Table S3 gives the success condition of each task, checked at every physics step; an interrupted continuous condition resets its counter. An episode ends at success, at a failure such as unsafe contact, leaving the workspace, or a drop, or at the time limit.

Roles and task groups. Table S3 names the target of each task. A moving receptacle is the target because its pose decides containment; in the passage tasks, the wire entry, which the wand must match within 30 mm, is the target, and the rest of the wire is context that must not be touched. The two task groups of Fig. 5 were fixed before the ablations: a task is rotation-sensitive if its target rotates during the lead and its orientation matters at contact, which excludes the balls, and context-active if it has an obstacle clearance m or a driver (a conveyor, a ramp, or a striker).

B.2. Scenarios, Demonstrations, and Splits

Each scene samples the target's initial state and motion parameters uniformly within task-specific ranges: besides the speeds of Table S3, harmonic amplitudes of up to 35 mm at 0.25–0.7 Hz, yaw rates of up to 1.2 rad/s, wire rotation rates of 2.5–7.5°/s, and ramp slopes of 5–15°. An occluded scene shows the target for at least eight initial frames, hides it for 0.5–3.0 s, and shows it again for at least three frames. Only physically feasible scenes are kept, and evaluation replays the stored scenes.

Table S3. **The fourteen ConMoto tasks.** Target: the body whose pose the policy receives (Sec. 3.1). Speed: target speed range (capture tasks: at the ramp exit). Every task requires its stages in order without a terminal failure; a grasp requires bilateral fingerpad contact, a lift of at least 0.05 m, and a 0.5 s hold. Limit: episode time limit. Groups of Fig. 5: R, rotation-sensitive; C, context-active (Appendix B.1). The released task configurations contain the exact thresholds and scenario ranges.

Task (code)	Target	Speed (m/s)	Success	Limit (s)	Group
Harmonic grasp (grasp)	block	0.03–0.20	Grasp the block moving harmonically and yawing, lift, and hold.	8	R
Conveyor grasp (t1_belt)	part	0.04–0.10	Grasp the part on the conveyor, lift, and hold.	8	C
Translating grasp (t1_xyz)	part	0.04–0.15	Grasp the translating part, lift, and hold.	8	–
Rotating grasp (t1_so3)	part	0.04–0.15	Grasp the translating and rotating part ($\|\omega\| \leq 1.2$ rad/s), lift, and hold.	8	R
Occluded grasp (t1_occ)	part	0.04–0.10	Grasp the part on the conveyor, which a canopy hides for 0.5–3.0 s, without unsafe contact with the canopy; lift and hold.	8	C
Moving receptacle (t2_trans)	receptacle	0.03–0.06	Place the held object fully inside the moving receptacle and release it to rest.	10	–
Rotating receptacle (t2_se3)	T-shaped receptacle	0.03–0.06	Place the held T-shaped part fully inside the translating and rotating T-shaped receptacle and release it to rest.	10	R
Wire passage (t3_phase)	wire entry	[X]	Align the held wand's loop with the rotating wire within 30 mm, then pass it along the wire to the end without contact.	21	R, C
Full wire passage (t3_full)	wire entry	[X]	Grasp the wand, complete the passage, exit the wire, and release the wand.	32	R, C
Ball capture (t4_sphere)	ball	[0.4–0.7]	Catch the ball after it leaves the ramp and before it touches the tray, lift, and hold.	6	C
Ball capture on table (t4_sphere_tabletop)	ball	[0.4–0.7]	Grasp the ball while it still rolls on the table after the ramp, lift, and hold.	6	C
Slider capture (t4_slider)	slider	[0.4–0.8]	Grasp the block while it still slides on the table after the ramp, lift, and hold.	6	C
Can capture (t4_can)	can	[0.4–0.7]	Grasp the can while it still rolls on the table after the ramp, before it tips over, lift, and hold.	6	R, C
Post-impact grasp (t5_replan)	part	0.12–0.14	After a striker deflects the part on the conveyor, grasp it along its grasp axis, lift, and hold.	6	R, C

The expert is a task-specific closed-loop state machine with privileged simulator state: the target's pose and velocity, the robot state, the task geometry, and true contact and stage events. It extrapolates the target and then tracks, descends, closes, and lifts in the grasping tasks, predicts the interception point in the capture tasks, and compensates the wire's phase in the passage tasks; its commands are clipped to 0.05 m and 0.5 rad per axis. A demonstration is kept if the expert succeeds, a replay of its actions reproduces the episode, and its path and jerk pass quality checks. The training pool holds 11,200 demonstrations, 800 per task, recorded at 20 Hz with RGB-D, robot states, commands, and simulator ground truth; policies receive only the inputs of Sec. 3.

We select configurations on 1,400 development scenes (100 per task) and report all ConMoto results on 1,400 held-out scenes, generated with the same ranges from seeds used in no training, validation, or development episode. Each task's held-out set holds its first 100 feasible scenes in seed order and is shared by all methods.

B.3. Real-Time Evaluation

Executor. All methods share one executor on the 50 ms control clock. Command j of query t is meant for $\tau_t + j\Delta_a$, and its execution time (Sec. 3.1) is

$$\tau^{\mathrm{a}}_{t,j} = \tau_t + j\Delta_a + d_{\mathrm{task}}, \tag{S1}$$

where $d_{\mathrm{task}} = 50$ ms for the post-impact grasp, which applies each command 50 ms after it is issued, and $d_{\mathrm{task}} = 0$ otherwise. As in RTC [2], latency counts in whole control periods: a query with total latency L_t hands over its chunk at the first boundary at or after it ends, $k_t = \lceil L_t/\Delta_a \rceil$ periods after capture, and execution continues from command k_t, the one meant for that boundary. The policy then observes again and starts the next query; meanwhile the previous chunk continues, so a policy replans as often as its latency allows. With $L_t = 0$, the next query starts one period after capture. A chunk holds the method's native number H_m of commands (Table S4) and is dropped if $k_t \geq H_m$. Before the first chunk arrives and whenever a chunk runs out, the controller holds the current arm target and gripper command; no command is repeated. Every command of FoveaWAM executes within $(H-1)\Delta_a + d_{\mathrm{task}}$ of capture, inside its forecast horizons.

Protocols. In real time, physics advances during each query by the method's latency, drawn per query from its distribution measured on one RTX 5090. With paused physics,

inference takes no physics time and a policy replans every 50 ms; this control removes the wait for inference, which also raises the feedback rate, and is not an upper bound. Equal latency charges every method the p95 latency of FoveaWAM; original executors let each baseline keep the execution scheme of its paper, such as temporal aggregation for ACT or streaming for DynamicVLA. The latency sweep adds 0 to 200 ms in steps of 50 ms after inference in real time. Rollouts run on RTX PRO 6000 and replay the latency measured on RTX 5090.

Latency measurement. We time each method alone on one RTX 5090 (32 GB), the GPU of our robot, at batch size 1 and in the precision of its release (FP32 for FoveaWAM), over three rounds of 30 warm-up and 300 measured calls on recorded observations with the required causal history. A timed query starts from the newest observation and ends when its first command is available. It includes all computation that needs the newest frame, such as preprocessing, image encoding, the online flow and motion descriptors of PUMA and DynamicWAM, sampling, and action decoding, and on the robot also tracking. The forecast heads of FoveaWAM, skipped at deployment, are timed separately. A query misses its deadline when this time-to-action exceeds 50 ms, and the action age of a command is its execution time minus the capture time. Appendix C.2 reports the measured values.

B.4. Metrics and Statistics

The task-balanced success rate of method m is

$$\mathrm{SR}(m) = \frac{100}{14} \sum_{k=1}^{14} \frac{1}{n_k} \sum_{i=1}^{n_k} y_{mki}, \tag{S2}$$

with binary outcomes y_{mki} on scene i of task k. The position error at first contact e_p is the distance of the end effector from the expert's contact frame on the target; in the placement tasks, which start with the object held, it uses the contact of the held object with the receptacle and the expert's placement pose. e_p averages over the 13 tasks with a contact, excluding `t3_phase`, and over the episodes that make contact; failures without contact remain in the SR denominator.

Paired intervals. Every method is trained once. Paired 95% intervals bootstrap held-out scenes within each task, jointly across methods, and quantify scene uncertainty for the trained weights; we treat differences whose interval lies within ± 2 points as ties. For the ablations, we also repeat the comparison without the phase-only wire passage (`t3_phase`, Appendix C.4).

B.5. Baselines and Ablation Controls

Common conditions. Each baseline keeps its official architecture, chunk length, initialization, and training recipe (Table S4), trains once on the same 11,200 demonstrations mapped to its action space through the same Cartesian controller, and is configured on development scenes. Rollouts use the agent view each baseline was trained with, any wrist slot masked, and the histories of Table 1; latency is measured in each method's native deployment setting. For the seven configurations timed both with and without a wrist view, the two differ by at most 3.8 ms in p95 latency, under a tenth of the control period; PUMA, OpenWAM with official acceleration, and DynamicWAM+SE(3) were timed only in their deployment setting. DP uses its authors' real-robot DDIM sampler with 16 steps, and OpenWAM is timed with and without its official cache and compilation.

Table S4. **Baseline training.** All methods train on the same 11,200 demonstrations with the official recipe of each method and run under the shared executor of Appendix B.3. Updates × batch: training budget. H: commands per returned chunk. Params: trainable parameters, frozen encoders excluded. DOM ckpt.: the released DynamicVLA checkpoint trained on DOM. OpenWAM-α generates video and actions jointly with video decoding disabled.

Method	Initialization	Updates×batch	H	Params
ACT [17]	scratch	50,000 × 32	100	[X]
DP [4]	scratch	50,000 × 32	15	[X]
$\pi_{0.5}$ [1]	`pi05_base`	30,000 × 256	10	[X]
$\pi_{0.5}$+SE(3)	`pi05_base`	30,000 × 256	10	[X]
DynamicVLA [16]	DOM ckpt.	50,000 × 44	20	[X]
PUMA [5]	official	[X]×[X]	[X]	[X]
OpenWAM-α [14]	official	34,490 × 192	32	[X]
DynamicWAM [10]	official	40,000 × 128	16	[X]
DynamicWAM+SE(3)	official	40,000 × 128	16	[X]
FoveaWAM	scratch	51,740 × 512	8	523.8M

SE(3)-augmented baselines. $\pi_{0.5}$+SE(3) and DynamicWAM+SE(3) receive the eight target-state tokens of Appendix A.1 from the same pose source, with the same validity flags. A two-layer MLP with a zero-initialized output layer maps them to the model's width; for $\pi_{0.5}$ they follow the image tokens in the prefix, where the action expert attends to them, and for DynamicWAM they join its kinematic descriptors. Training follows the base model's recipe.

Ablation controls. The five controls of Sec. 4.3 each change one component of the configuration of Table S2 and keep the data, the recipe with its checkpoint selection, and the executor. *No dedicated SE(3)* removes $\mathcal{L}_P$ and the pose head, so that the target receives only the uniform coarse scene-flow supervision of the whole scene; *no scene-flow supervision* removes $\mathcal{L}_F$ and the flow readout; both keep the target-state tokens and all auxiliary objectives. *No target history* removes target-state slots 0–6, the past poses with their velocity and acceleration features, and keeps slot 7, the current pose (Appendix A.1). *No future supervision* removes

$\mathcal{L}_P$, $\mathcal{L}_F$, and the three future appearance losses with their heads, and keeps the current-target reconstruction. *No auxiliary objectives* removes $\mathcal{L}_{\text{aux}}$, the three future appearance losses and the current-target reconstruction, and keeps $\mathcal{L}_P$ and $\mathcal{L}_F$. Since every forecast head is skipped at deployment, only the no-history control changes the inputs at inference.

C. Additional Simulation Results

C.1. Per-Task Results

Table S5 breaks the held-out ConMoto results of Table 1 down by task for every method, with the paired interval of FoveaWAM against the strongest other method on each task; its last two rows give the position error at first contact e_p and the number of episodes with a contact (Appendix B.4). With the real-robot DDIM16 sampler, Diffusion Policy reaches 30.7% SR in real time and 31.0% with physics paused. OpenWAM with cache and compile disabled has no contact in some tasks, so its full 13-task e_p is undefined. [Tasks on which every method fails or a baseline outperforms FoveaWAM, and the dominant failure of FoveaWAM.]

C.2. Evaluation Protocols and Latency

Table S6 evaluates every method under three further protocols. Paused physics shows how the relative performance changes when the cost of computation is ignored, equal latency compares all methods at the p95 latency of FoveaWAM, and the original executors check whether the shared executor removes a capability that a baseline relies on. With paused physics, the difference between FoveaWAM and [name] is [±X.X] points, and under equal latency it is [±X.X] points. The original executors change the SR of the baselines by at most [X.X] points. Table S7 reports five added-latency points for nine configurations; PUMA and FoveaWAM remain pending. Fig. 4 reads this table directly and plots the selected comparisons; all methods remain in the table.

Table S6. **Protocol controls** on held-out ConMoto scenes (SR, %) under the protocols of Appendix B.3; real time is the protocol of Table 1. Late: queries in real time whose time-to-action exceeds the 50 ms control period (%). Age: p95 action age in real time (ms); –: no command executed. Blue/bold and light blue rank the reported values only; pending entries are excluded.

Method	Paused	Equal latency	Original executor	Real time	Late (%)	Age p95
ACT	40.6	—	—	41.6	0.0	50
Diffusion Policy	31.0	—	—	30.7	100.0	150
$\pi_{0.5}$	66.0	—	—	**53.8**	100.0	150
$\pi_{0.5}$ + SE(3)	**74.9**	—	—	52.8	100.0	150
DynamicVLA	54.6	—	—	50.9	100.0	150
PUMA	—	—	—	—	—	—
OpenWAM-α	58.3	—	—	46.1	100.0	350
OpenWAM-α (off)	58.3	—	—	15.8	100.0	950
DynamicWAM	49.1	—	—	40.5	100.0	250
DynamicWAM + SE(3)	64.1	—	—	36.8	100.0	350
FoveaWAM	—	—	–	—	—	—

Table S7. **Success versus added latency** on 1,400 held-out ConMoto scenes per point (SR in %, real-time protocol, one training seed). The 0 and 150 ms columns equal the Native and +150 ms columns of Table 1. Blue/bold and light blue rank the reported values only; pending entries are excluded.

	Added latency (ms)				
Method	0	50	100	150	200
ACT	41.6	43.5	36.5	24.3	16.5
Diffusion Policy	30.7	26.1	21.0	15.9	14.3
$\pi_{0.5}$	**53.8**	**47.2**	**39.6**	27.8	13.6
$\pi_{0.5}$ + SE(3)	52.8	39.8	32.4	**28.2**	15.1
DynamicVLA	50.9	46.4	30.6	23.4	17.8
PUMA	—	—	—	—	—
OpenWAM-α	46.1	36.1	27.9	24.7	**22.1**
OpenWAM-α (off)	15.8	13.7	14.2	10.4	9.5
DynamicWAM	40.5	27.6	25.0	20.6	20.9
DynamicWAM + SE(3)	36.8	30.6	23.9	21.9	21.3
FoveaWAM	—	—	—	—	—

Timing. Table S8 breaks the time-to-action of FoveaWAM on one RTX 5090 down by stage, measured as in Appendix B.3; Table 1 gives the p95 latency of every method and Table S4 the parameter counts. On the robot, we report the p50, p95, and maximum action age and the fraction of all queries of the test trials that miss the 50 ms deadline, together with the peak memory.

Table S5. **Per-task held-out success** (%) on ConMoto for every method of Table 1, real-time protocol. DP: Diffusion Policy. DVLA: DynamicVLA. OWAM: OpenWAM-α with official cache/compile; OW-off: both disabled. DWAM: DynamicWAM. +: with our SE(3) history tokens. σ: FoveaWAM with nominal synthetic correlated pose noise (Appendix C.5). Δ: FoveaWAM minus the strongest other method on that task, with its 95% paired interval. The last two rows give the position error at first contact e_p (mm), averaged over the 13 tasks with a contact, and the number n of episodes with a contact (Appendix B.4); –: no 13-task mean. Task codes are listed in Table S3. Shading of Δ: blue for higher SR, orange for lower, saturating at 15 points. Blue/bold and light blue rank the reported values only; pending entries are excluded.

Task	ACT	DP	$\pi_{0.5}$	$\pi_{0.5}$+	DVLA	PUMA	OWAM	OW-off	DWAM	DWAM+	σ	**Ours**	Δ [95% CI]
Harmonic grasp	49.0	31.0	51.0	**85.0**	39.0	—	41.0	1.0	50.0	51.0	—	—	—
Conveyor grasp	45.0	45.0	**96.0**	58.0	89.0	—	62.0	0.0	59.0	47.0	—	—	—
Translating grasp	14.0	33.0	59.0	38.0	**61.0**	—	53.0	0.0	53.0	15.0	—	—	—
Rotating grasp	24.0	20.0	32.0	**44.0**	26.0	—	42.0	3.0	30.0	37.0	—	—	—
Occluded grasp	30.0	28.0	37.0	51.0	**80.0**	—	34.0	0.0	44.0	36.0	—	—	—
Moving receptacle	26.0	8.0	26.0	19.0	**39.0**	—	28.0	0.0	0.0	0.0	—	—	—
Rotating receptacle	**0.0**	**0.0**	**0.0**	**0.0**	**0.0**	—	**0.0**	**0.0**	**0.0**	**0.0**	—	—	—
Wire passage	99.0	99.0	**100.0**	**100.0**	99.0	—	50.0	58.0	23.0	23.0	—	—	—
Wire passage, full	0.0	3.0	1.0	5.0	**12.0**	—	0.0	0.0	0.0	0.0	—	—	—
Ball capture	36.0	16.0	**46.0**	21.0	41.0	—	45.0	13.0	24.0	23.0	—	—	—
Ball capture, table	71.0	81.0	**100.0**	**100.0**	88.0	—	99.0	88.0	98.0	**100.0**	—	—	—
Slider capture	56.0	30.0	58.0	68.0	28.0	—	**71.0**	31.0	47.0	63.0	—	—	—
Can capture	91.0	31.0	98.0	99.0	72.0	—	**100.0**	5.0	96.0	**100.0**	—	—	—
Post-impact grasp	42.0	5.0	49.0	**51.0**	39.0	—	21.0	22.0	43.0	20.0	—	—	—
Task-balanced	41.6	30.7	**53.8**	52.8	50.9	—	46.1	15.8	40.5	36.8	—	—	—
e_p (mm)	32.2	36.2	31.2	32.5	**30.0**	—	31.7	–	31.2	37.0	—	—	–
n	1110	941	1231	1187	1203	—	1115	439	1193	970	—	—	–

Table S8. **Latency breakdown** of one query of FoveaWAM on one RTX 5090 at batch size 1 (ms, p50 / p95), and action age on the robot.

Stage	p50	p95
Pose tracking (per frame, robot)	—	—
Image encoding	—	—
World stream (once per query)	—	—
Action stream (5 Euler steps)	—	—
Time-to-action (total)	—	—
Forecast heads (training only, for reference)	—	—
Action age at execution (robot)	—	—
Queries over 50 ms (robot, %)	—	

C.3. Results on DOMINO

On DOMINO [5], we compare FoveaWAM, $\pi_{0.5}$, DynamicWAM, and PUMA on 24 single-arm Franka tasks of its clean level-1 setting. All methods use the benchmark's demonstrations, scenes, and native evaluator, one training seed, and 50 episodes per task and method. We report SR and the manipulation score (MS), which scales route completion by penalties for leaving the workspace or view and for collisions (Table 2). FoveaWAM uses domain-specific normalization and reads the simulator pose of the target named by the instruction; slots without a valid pose are masked as in Appendix A.1. The RGB-based baselines do not receive this pose.

Data and adaptation. The baselines share 1,200 training and 240 validation demonstrations, retain all original captures, and execute absolute joint-position targets with DOMINO's native controller. No extra joint-difference filter or policy-side clipping is applied. DynamicWAM retains its official stride-3, horizon-16 sampler and requires at least 10 complete windows per demonstration: 1,054 training and 210 validation sources contribute supervision. The short sources remain in the corpus; two tasks have no eligible training source and one has only four. All 24 tasks remain in the test set. $\pi_{0.5}$ and PUMA use all 1,200/240 sources with their own window rules. All normalizers use training data only.

DynamicWAM adapts the published model's state/action projections to eight dimensions and trains for 40,000 updates at global batch 128; evaluation uses the final checkpoint. $\pi_{0.5}$ starts from its official base and trains for 30,000 updates at global batch 256; evaluation uses its final EMA. PUMA retains the published shared weights, initializes an eight-dimensional output projection, and adapts all originally trainable parameters while keeping its teacher frozen, at global batch 64 on 16 GPUs. Its prespecified budget is 30,000 updates, extendable to 40,000 only with continued validation improvement. Every 2,000 updates, it evaluates all 240 validation sources (18,618 windows), using task-macro normalized action MSE. After at least 10,000 updates, three

consecutive checks without a 0.5% relative improvement stop training. Training stopped at 12,000; the lowest validation MSE selected update 6,000. Checkpoint selection and stopping never use test outcomes. The comparison shares data sources and test scenes while retaining method-specific training recipes; it does not match optimization steps or compute across methods.

Results. All three baselines complete the same 1,200 fixed scenes on 32 GPUs, with 25 scenes per arm and task, no missing or duplicate outcomes, and every failure retained. DynamicWAM succeeds in 352 scenes, PUMA in 220, and $\pi_{0.5}$ in 208. DynamicWAM leads on Track (34.9% SR), whereas $\pi_{0.5}$ leads on Intercept (33.8%); PUMA reaches 12.8% and 29.5%, respectively. All averages weight the 24 tasks equally, rather than averaging the two unequal-size task groups. These are single-training-seed results; they do not establish robustness across training seeds. [Add the accepted DOMINO comparison for FoveaWAM.]

C.4. Final Ablations

Table S9 reports the five controls of Sec. 4.3 (Appendix B.5) with native latency and with 150 ms of added latency, each with its paired difference from the full model. The last two columns repeat the paired differences without the phase-only wire passage (`t3_phase`): on development scenes, retraining the same configuration with different seeds changed SR by up to [7.4] points, mostly on this task, whose success ranged from [0] to [71]% (Appendix B.4). The two task groups of Fig. 5 are listed in Appendix B.1. [Whether any conclusion of Sec. 4.3 changes without `t3_phase`.]

C.5. Robustness to Pose Noise

With frozen weights, we evaluate three synthetic perturbations of the target pose: nominal temporally correlated noise, twice its amplitude, and nominal noise with contiguous missing observations. The nominal process combines a fixed per-episode bias and a stationary correlated component, each with per-axis standard deviations of 5 mm and 1°, with a correlation time of 200 ms. Dropouts last 200 ms, separated by geometrically distributed valid gaps with a mean of 3.8 s, giving a long-run missing fraction of 5%. The perturbation applies to the current and past estimates before the history features are computed.

Fig. S1 evaluates the final model under these perturbations, on the same held-out scenes at zero added latency. Relative to clean poses, SR changes by [X.X], [X.X], and [X.X] points under nominal correlated noise, twice its amplitude, and missing observations.

Figure placeholder: Fig. S1 — pose noise

x: clean poses, nominal correlated noise, twice its amplitude, and nominal noise with missing observations; y: SR of FoveaWAM per task family, frozen weights, real-time protocol; paired intervals over matched scenes.

Figure S1. **Robustness to pose noise.** SR of FoveaWAM under synthetic perturbations of the target pose, relative to clean poses.

C.6. Forecast Diagnostics

Table S10 compares the learned SE(3) forecast of the final model offline with constant-velocity and constant-acceleration extrapolation from the same pose history.

Table S10. **Offline prediction errors.** Target-position error at 400 ms (mm) on the same held-out observations. CV and CA are analytic extrapolations from causal pose history; the learned head reads frozen full-model states.

	Const. vel.	Harmonic	Roll/slide	Impact
CV extrapolation	—	—	—	—
CA extrapolation	—	—	—	—
Learned pose head	—	—	—	—

Table S9. **Final ablations** on held-out ConMoto scenes (Appendix C.4). SR (%) with native latency and with 150 ms of added latency; Δ: control minus FoveaWAM (points) with its paired 95% interval. Last two columns: Δ without the phase-only wire passage (`t3_phase`), 13 tasks. Shading of Δ: blue for higher SR, orange for lower, saturating at 15 points.

	Native		+150 ms		Δ w/o `t3_phase`	
Variant	SR	Δ [95%]	SR	Δ [95%]	Native	+150 ms
FoveaWAM *	—	–	—	–	–	–
No dedicated SE(3) (uniform coarse scene flow)	—	—	—	—	—	—
No scene-flow supervision	—	—	—	—	—	—
No target history	—	—	—	—	—	—
No future supervision	—	—	—	—	—	—
No auxiliary objectives	—	—	—	—	—	—

D. Real-Robot Experiments

D.1. Platform and Tasks

The robot is a Franka arm ([model]) with [gripper model] and a fixed RealSense D435 RGB-D camera, commanded through [a timestamped Cartesian interface] at [control frequency]; a fixed third-person camera records RGB-D at [resolution and frame rate], calibrated by [procedure] before each session. FoveaWAM uses this view, baselines whose recipes use a wrist view also receive [wrist camera], and all policies run on one RTX 5090. The task devices are installed on this platform one at a time (Fig. S2; Table S11). The turntable block of T3 has unequal sides and must be grasped across its short sides, so its orientation at contact matters; the toy car of T4 runs at a fixed throttle.

Figure placeholder: Fig. S2 — real-robot platform

Content: one unretouched photo of the shared platform from behind the robot with one task device installed; the third-person RGB-D camera, the robot, and the wrist camera if a baseline uses one, each labeled.

Figure S2. **Real-robot platform** with one task device installed.

D.2. Pose Tracking and Labels

Tracking is marker-free and uses one metric mesh per object instance; a ball is defined by its center and radius. In the first frame, Grounding DINO [9] localizes the object named by the task, SAM 2.1 [11] segments it, and FoundationPose [15] registers its pose; afterwards, Cutie [3] propagates the mask and FoundationPose refines a Kalman-filter prediction at each update. An empty mask makes the estimate invalid rather than carrying the previous pose forward, and [re-initialization after a loss]. The tracker updates on the newest frame at every control-period boundary and keeps that frame's capture time, from which the target-state tokens take their finite differences; tracking is part of the measured latency (Table S8).

The target-state inputs of a demonstration replay the same tracker causally over its recorded RGB-D. The pose labels are the tracked poses at future boundaries, [refined offline over the episode], and the flow labels follow [3D point tracks of the recorded RGB-D, as for DROID (Appendix A.3)]. For the ball, the target-state tokens and the pose head use translation only. Table S13 reports how often tracking was lost in the test trials.

Table S13. **Tracking on the real robot.** Per task, over the test trials of FoveaWAM: control-period boundaries, the fraction with an invalid estimate, the longest gap, and re-initializations.

Task	Boundaries	Invalid (%)	Longest gap (s)	Re-init.
T1	—	—	—	—
T2	—	—	—	—
T3	—	—	—	—
T4	—	—	—	—
T5	—	—	—	—
T6	—	—	—	—

D.3. Training and Evaluation Protocol

FoveaWAM is fine-tuned per task from the DROID base (Appendix A.3). $\pi_{0.5}$, DynamicWAM, and ACT [17] train or fine-tune on the same demonstrations with the initialization, recipe, and chunk length of Table S4; DynamicWAM executes [with or without real-time chunking]. Demonstrations, development trials, and test trials come from separate sessions.

Each task has 20 pre-generated trial keys that fix the object instance, its initial pose, and the device setting. For each key, the four methods run consecutively in a random order, so that drift of the apparatus does not align with the methods, after an operator resets the scene against a reference image. A trial ends at success, failure, a safety stop, or a time limit of about three times the task's median demonstration duration. A key is repeated only if the device deviates from it on its own ([speed outside ±X% of the setting or a device fault]); everything else, including a tracking loss of FoveaWAM,

Table S11. **Real-robot tasks.** Target: the tracked object. Trial key: what each of the 20 test keys fixes. Speed: target speed measured from the recorded RGB-D of all test trials (m/s, mean ± s.d.). A grasp holds the object between both fingers. Each task has 100 training and 20 validation demonstrations.

Task	Target	Motion; trial key	Speed	Success	Limit (s)
T1	Block	Belt conveyor; [placement on the belt, belt speed]	—	Grasp the block and lift it by [X] cm.	[X]
T2	Open box	Belt conveyor; [box placement, belt speed]; the block starts in the gripper	—	Release the block into the box; it stays inside [until the box leaves the workspace].	[X]
T3	Block, unequal sides	Turntable; [radius, initial angle and yaw of the block, angular speed]	—	Grasp the block across its short sides and lift it by [X] cm.	[X]
T4	Toy car	Remotely driven at a fixed throttle; [start pose, heading]	—	Grasp the car and lift it by [X] cm.	[X]
T5	Tennis ball	[ball-motion mechanism, release position and motion setting]	—	Grasp the tennis ball and lift it by [X] cm.	[X]
T6	Block on cart	Cart translates down an inclined rail; [slope, release point]	—	Grasp the block and lift it by [X] cm.	[X]

Table S12. **Real-robot successes and failures.** Top: successes per task (of 20 trials; tasks in Table S11); All: pooled success rate (%) with its 95% Wilson interval; Δ: paired difference from FoveaWAM (points) with a bootstrap interval over trial keys. Bottom: failures by category over all 120 trials of each method (Appendix D.4); –: not applicable. Shading of Δ: blue for higher SR, orange for lower, saturating at 15 points.

Method	T1	T2	T3	T4	T5	T6	All	Δ
$\pi_{0.5}$	—	—	—	—	—	—	—	—
DynamicWAM	—	—	—	—	—	—	—	—
ACT	—	—	—	—	—	—	—	—
FoveaWAM	—	—	—	—	—	—	—	–

	Tracking	Late	Misaligned	Slip/drop	Collision	Timeout	Failures
$\pi_{0.5}$	–	—	—	—	—	—	—
DynamicWAM	–	—	—	—	—	—	—
ACT	–	—	—	—	—	—	—
FoveaWAM	—	—	—	—	—	—	—

counts as a failure. We report successes per task and the pooled success rate with 95% Wilson intervals, and paired differences from a bootstrap over trial keys within each task. End-to-end time-to-action is logged for every query on the same RTX 5090, including tracking for FoveaWAM.

D.4. Detailed Results and Failures

Table S12 reports the successes per task with intervals and paired differences, and the failures of each method by category. The categories, fixed before the test trials, are observable events: tracking lost (FoveaWAM only), late (the gripper closes behind the target), misaligned grasp, slip or drop after contact, collision or safety stop, and timeout. [Tasks with the largest margin and their failure categories; the most frequent failures of FoveaWAM.]

References

[1] Kevin Black, Noah Brown, James Darpinian, et al. $\pi_{0.5}$: a vision-language-action model with open-world generalization. In *Proceedings of The 9th Conference on Robot Learning*, pages 17–40, 2025. 4

[2] Kevin Black, Manuel Galliker, and Sergey Levine. Real-time execution of action chunking flow policies. In *Advances in Neural Information Processing Systems*, pages 33383–33407, 2025. 3

[3] Ho Kei Cheng, Seoung Wug Oh, Brian Price, Joon-Young Lee, and Alexander Schwing. Putting the object back into video object segmentation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024. 8

[4] Cheng Chi, Siyuan Feng, Yilun Du, Zhenjia Xu, Eric Cousineau, Benjamin C. M. Burchfiel, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. In *Proceedings of Robotics: Science and Systems*, 2023. 4

[5] Heng Fang, Shangru Li, Shuhan Wang, Xuanyang Xi, Dingkang Liang, and Xiang Bai. Towards generalizable robotic manipulation in dynamic environments. In *European Conference on Computer Vision*, pages 314–333, 2026. 4, 6

[6] Kaiming He, Xinlei Chen, Saining Xie, Yanghao Li, Piotr Dollár, and Ross Girshick. Masked autoencoders are scalable vision learners. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 16000–16009, 2022. 1

[7] Wenlong Huang, Yu-Wei Chao, Arsalan Mousavian, Ming-Yu Liu, Dieter Fox, Kaichun Mo, and Li Fei-Fei. PointWorld: Scaling 3D world models for in-the-wild robotic manipulation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 20765–20779, 2026. 2

[8] Alexander Khazatsky, Karl Pertsch, Suraj Nair, et al. DROID: A large-scale in-the-wild robot manipulation dataset. In *Proceedings of Robotics: Science and Systems*, 2024. 2

[9] Shilong Liu, Zhaoyang Zeng, Tianhe Ren, Feng Li, Hao Zhang, Jie Yang, Qing Jiang, Chunyuan Li, Jianwei Yang, Hang Su, Jun Zhu, and Lei Zhang. Grounding DINO: Marrying DINO with grounded pre-training for open-set object detection. In *European Conference on Computer Vision*, 2024. 8

[10] Yunfan Lou, Hewen Gao, Xiyu Zhu, Zhuoran Qiao, Xuan Han, Yifan Yang, Yifan Ye, Boxian Yao, and Zhibo Pang. DynamicWAM: Dual-path motion conditioning for world-action models in dynamic manipulation. *arXiv preprint arXiv:2608.00793*, 2026. 4

[11] Nikhila Ravi, Valentin Gabeur, Yuan-Ting Hu, Ronghang Hu, Chaitanya Ryali, Tengyu Ma, Haitham Khedr, Roman Rädle, Chloe Rolland, Laura Gustafson, Eric Mintun, Junting Pan, Kalyan Vasudev Alwala, Nicolas Carion, Chao-Yuan Wu, Ross Girshick, Piotr Dollár, and Christoph Feichtenhofer. SAM 2: Segment anything in images and videos. In *International Conference on Learning Representations*, pages 28085–28128, 2025. 8

[12] Oriane Siméoni, Huy V. Vo, Maximilian Seitzer, Federico Baldassarre, Maxime Oquab, Cijo Jose, Vasil Khalidov, Marc Szafraniec, Seungeun Yi, Michaël Ramamonjisoa, et al. DINOv3. *arXiv preprint arXiv:2508.10104*, 2025. 1

[13] Emanuel Todorov, Tom Erez, and Yuval Tassa. MuJoCo: A physics engine for model-based control. In *IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 5026–5033, 2012. 2

[14] Yuran Wang, Siqiao Huang, Mingleyang Li, Chenhao Zhang, Jiaqi Liang, Weiyang Jin, Yue Chen, Xuemin Chi, Donghao Zhou, Qize Yu, Yu-Kai Wang, Yuhan Rui, Shenzhe Yao, Zhen Yuan, Zhenhao Shen, Kefei Zhu, Zijie Zhu, Ning Gao, Xiaowei Chi, Guanqi He, Shanghang Zhang, Hao Dong, Lin Shao, and Hang Zhao. OpenWAM: An open, modular exploration towards systematic world-action model pretraining. *arXiv preprint arXiv:2609.07398*, 2026. 4

[15] Bowen Wen, Wei Yang, Jan Kautz, and Stan Birchfield. FoundationPose: Unified 6D pose estimation and tracking of novel objects. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 17868–17879, 2024. 8

[16] Haozhe Xie, Beichen Wen, Jiarui Zheng, Zhaoxi Chen, Fangzhou Hong, Haiwen Diao, and Ziwei Liu. DynamicVLA: A vision-language-action model for dynamic object manipulation. In *Advances in Neural Information Processing Systems*, 2026. Accepted for publication. 4

[17] Tony Z. Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. Learning fine-grained bimanual manipulation with low-cost hardware. In *Proceedings of Robotics: Science and Systems*, 2023. 4, 8

[18] Yuke Zhu, Josiah Wong, Ajay Mandlekar, Roberto Martín-Martín, Abhishek Joshi, Kevin Lin, Abhiram Maddukuri, Soroush Nasiriany, and Yifeng Zhu. robosuite: A modular simulation framework and benchmark for robot learning. *arXiv preprint arXiv:2009.12293*, 2020. 2